

Here is the complete PRD for your Job Search Automation Platform:

Product Requirements Document (PRD)

Job Search Automation Platform — "JobPilot"

Version: 1.0

Author: @izaanibrahim37

Date: 2026-06-18

Status: Draft

Overview

JobPilot is a backend-driven job search automation platform that eliminates the manual effort of searching for jobs across multiple portals. A candidate fills a one-time Google Form with their profile and uploads their resume. The system then autonomously scrapes relevant job listings from multiple job portals (Naukri, LinkedIn, Glassdoor, etc.) at a configured frequency and delivers a consolidated Excel report directly to the candidate's email.

Goals & Objectives

- Reduce candidate time spent on manual job searching by 90%
 - Aggregate job listings from multiple portals into a single, clean report
 - Support configurable scraping frequency (daily, every 2 days, every 3 days, etc.)
 - Run multiple portal-specific agents simultaneously (parallel execution)
 - Require zero technical knowledge from the end user
-

Scope

In Scope:

- Google Form intake + Google Sheets storage
- Resume parsing agent
- Multi-portal scraping agents (Naukri, LinkedIn, Glassdoor as Phase 1)

- Scheduler / cron-based trigger system
- Email delivery of consolidated job report (Excel/CSV)
- Admin manual trigger capability

Out of Scope (Phase 1):

- Auto-applying to jobs on behalf of the candidate
- User dashboard or web UI
- Payment/subscription management
- ATS (Applicant Tracking System) integration

User Personas

Persona	Description
Job Seeker	Fills the Google Form once, receives periodic job reports
Admin/Operator	Manages the backend, triggers manual runs, monitors agent health

Functional Requirements

1. Candidate Intake (Google Form)

The form must collect the following fields:

- Full Name
- Email ID
- Phone Number
- Current Location / City
- Preferred Job Location(s) (multi-select)
- Current Job Designation / Title
- Years of Experience
- Skills (comma-separated or multi-select)
- Preferred Job Roles / Keywords
- Expected Salary (range or fixed)
- Notice Period
- Employment Type Preference (Full-time / Part-time / Contract / Remote)

- Job Portals to Search (Naukri / LinkedIn / Glassdoor / All)
- Report Frequency (Daily / Every 2 Days / Every 3 Days / Weekly)
- Resume Upload (PDF or DOCX)

All form responses must auto-populate into a linked **Google Sheet** with one row per candidate.

2. Backend Row Processor (Picker/Cron)

- A scheduled cron job runs at a configured interval (e.g., every 24 hours)
 - The picker reads the Google Sheet row by row
 - For each candidate row, it checks:
 - Is the candidate active?
 - Has the configured frequency interval elapsed since the last run?
 - If eligible, it dispatches the candidate's data to the orchestration pipeline
 - Supports **manual trigger** by admin for a specific candidate or all candidates
-

3. Resume Parsing Agent

- Accepts the uploaded resume file (PDF/DOCX) from Google Drive or Sheet attachment
 - Extracts structured data:
 - Name, contact info
 - Work experience (company, role, duration)
 - Education
 - Skills and technologies
 - Certifications
 - Projects
 - Merges parsed resume data with form-submitted data to build a **Candidate Profile Object**
 - Libraries/Tools: PyMuPDF / pdfplumber for PDF, python-docx for DOCX, or a third-party API like Affinda or Sovren
-

4. Job Portal Scraping Agents

Each portal has its own isolated agent. Agents run in **parallel** for a given candidate.

Common Agent Behavior:

- Accepts the Candidate Profile Object as input
- Constructs search queries based on: job role keywords, location, experience, employment type
- Scrapes/crawls the portal for matching job listings
- Deduplicates results
- Outputs a structured list of jobs

Fields collected per job listing:

- Job Title
- Company Name
- Location
- Job Type (Full-time / Remote / etc.)
- Experience Required
- Salary Range (if available)
- Date Posted
- Job Description Summary
- Application URL / Direct Link
- Source Portal

Agent Specifications:

Agent	Portal	Method
NaukriAgent	Naukri.com	Web crawler + scraper (Selenium / Playwright + BeautifulSoup)
LinkedInAgent	LinkedIn.com	Scraper (Playwright) or LinkedIn Jobs API
GlassdoorAgent	Glassdoor.com	Web scraper (Playwright + BeautifulSoup)
IndeedAgent (Phase 2)	Indeed.com	Scraper or Indeed Publisher API

Rate Limiting & Anti-bot Handling:

- Randomized request delays between scrapes
 - Rotating user-agent headers
 - Proxy rotation support (optional, Phase 2)
 - Headless browser with stealth plugins (e.g., `playwright-stealth`)
-

5. Aggregation & Report Generation

- Collects output from all parallel agents for a candidate
 - Merges all job listings into a single dataset
 - Removes duplicates across portals (match on Job Title + Company + Location)
 - Sorts by: Date Posted (newest first), then by relevance score
 - Generates an **Excel (.xlsx)** report with:
 - One sheet per portal (Naukri, LinkedIn, Glassdoor)
 - One consolidated "All Jobs" sheet
 - Columns: Job Title, Company, Location, Salary, Experience, Type, Posted Date, Apply Link, Source
 - Library: `openpyxl` or `pandas` + `xlsxwriter`
-

6. Email Delivery

- Sends the Excel report as an email attachment to the candidate's registered email
 - Email includes:
 - Candidate name
 - Report generation date
 - Total jobs found (per portal + total)
 - Direct apply links highlighted
 - SMTP integration (Gmail SMTP / SendGrid / AWS SES)
 - Retry logic on email failure (max 3 retries)
-

7. Scheduler & Trigger System

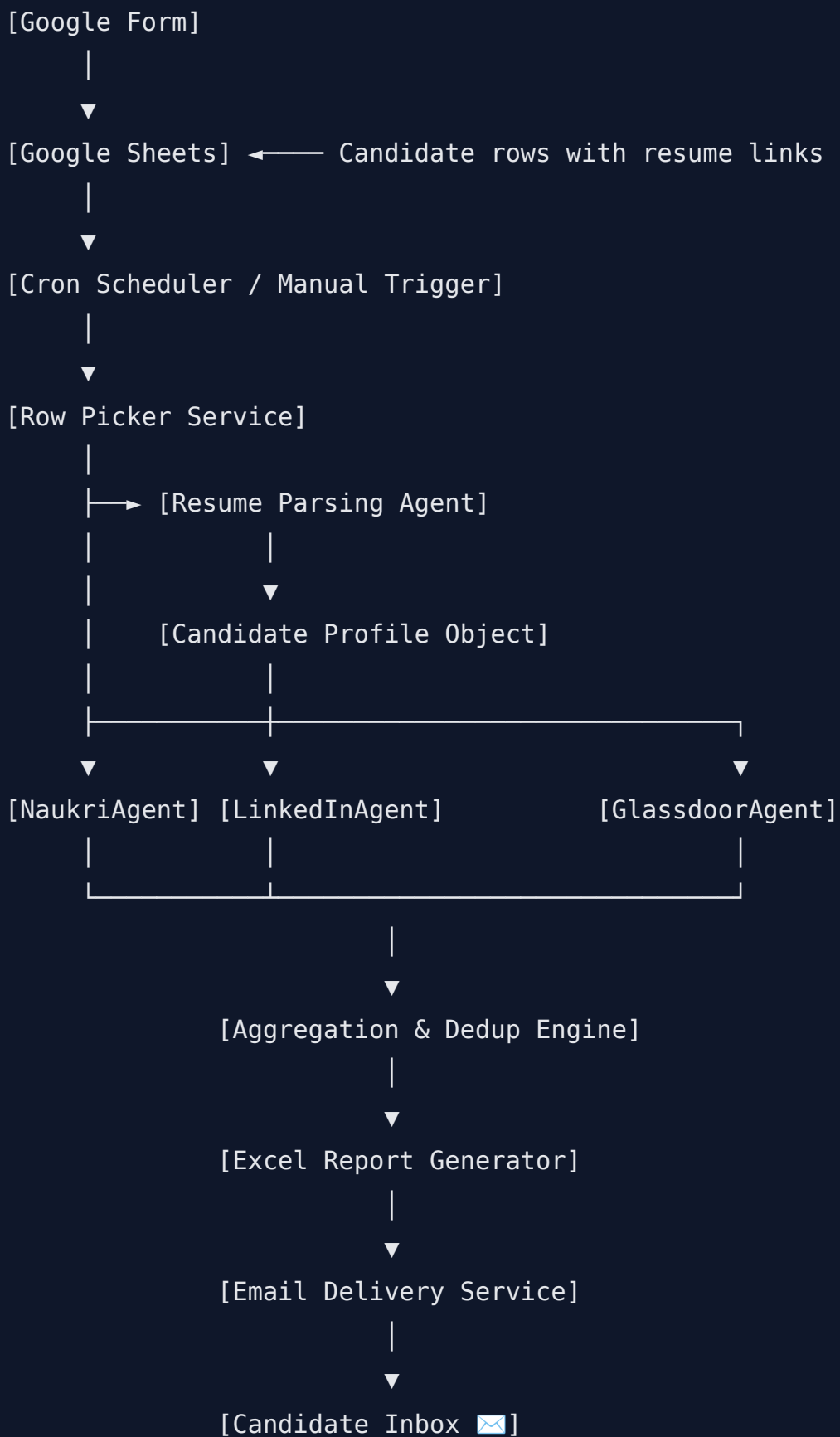
- **Cron-based scheduler** (e.g., runs every 6 hours to check eligibility)
- Per-candidate frequency stored in the Google Sheet (e.g., `frequency_days = 3`)

- Tracks `last_run_timestamp` per candidate row
 - **Manual Trigger API endpoint** (admin-only, secured with API key):
 - `POST /trigger/candidate/{row_id}` — trigger for one candidate
 - `POST /trigger/all` — trigger for all eligible candidates
 - Supports **concurrent agent execution** using Python `asyncio` or `concurrent.futures`
-

Non-Functional Requirements

Requirement	Detail
Scalability	Must handle up to 500 concurrent candidate pipelines
Reliability	99% uptime for the scheduler; retry on agent failure
Security	Google Sheet access via OAuth2 Service Account; API key auth for admin endpoints
Performance	Full pipeline per candidate (all agents) must complete within 10 minutes
Logging	Structured logs per candidate run (start, agent status, email status, errors)
Error Handling	Failed agents must not block report generation; partial reports are acceptable
Data Privacy	Resume data stored only in Google Drive; no PII stored in application DB

Technical Architecture



Tech Stack

Layer	Technology
Language	Python 3.11+
Scheduler	APScheduler / Celery Beat / GitHub Actions cron
Scraping	Playwright, BeautifulSoup4, Selenium (fallback)
Resume Parsing	pdfplumber, python-docx, or Affinda API
Google Integration	<code>google-api-python-client</code> , <code>gspread</code>
Report Generation	pandas, openpyxl, xlsxwriter
Email	smtplib / SendGrid SDK / AWS SES
Concurrency	asyncio / concurrent.futures / Celery workers
Hosting	AWS EC2 / GCP Cloud Run / Railway / Render
Logging	Python <code>logging</code> + structured JSON logs
Config Management	<code>.env</code> + <code>python-dotenv</code>

Data Flow Summary

1. Candidate fills Google Form → data saved to Google Sheet
2. Cron runs → Picker reads eligible rows
3. Resume Parsing Agent extracts structured profile from uploaded resume
4. Parallel agents (Naukri, LinkedIn, Glassdoor) scrape jobs based on profile
5. Results aggregated, deduplicated, sorted
6. Excel report generated with all jobs
7. Report emailed to candidate
8. `last_run_timestamp` updated in Google Sheet

Milestones & Phases

Phase	Deliverable	Timeline
-------	-------------	----------

Phase 1	Google Form + Sheet integration, Resume Parser, NaukriAgent, Email delivery	Week 1-3
Phase 2	LinkedInAgent, GlassdoorAgent, Parallel execution, Scheduler	Week 4-5
Phase 3	Admin manual trigger API, Logging dashboard, Error handling	Week 6
Phase 4	Proxy rotation, IndeedAgent, Relevance scoring, Report enhancements	Week 7-8

Risks & Mitigations

Risk	Mitigation
Portal blocks scraper (anti-bot)	Use stealth plugins, rotating proxies, randomized delays
Resume parsing inaccuracy	Fallback to form-submitted data; flag low-confidence parses
Google Sheets API rate limits	Batch reads; cache sheet data locally per run
Email delivery failure	Retry logic + fallback SMTP provider
Agent timeout	Per-agent timeout (5 min max); partial results still reported

Open Questions

1. Should candidates be able to update their profile after initial form submission?
2. Is there a need for an unsubscribe/pause mechanism for candidates?
3. Should the report include jobs the candidate has already seen in previous reports (dedup across runs)?
4. What is the maximum number of jobs per portal per report?
5. Should salary filtering be a hard filter or a soft preference?

This PRD covers the full scope of JobPilot. Would you like me to create this as a GitLab Epic with child issues for each phase, or generate the initial project scaffolding code?